# Smart Study Advisor

Lab 3 Report

Multi-Paradigm AI Course Recommendation System

## Team Members

| # | Name | Student ID |
|---|---|---|
| 1 | NourEldeen Mohamed Abbas | 23010920 |
| 2 | Ali Amr Abdallah | 23010589 |
| 3 | Omar Assem Ahmed | 23010610 |
| 4 | Mohamed Ahmed Slama | 23010716 |

# Contents


**1 Introduction** **3**

**2 System Architecture Overview** **3**
2.1 High-Level Architecture Diagram . . . 3
2.2 Technology Stack Summary . . . 4

**3 Architecture: Where Each Paradigm Is Used and Why It Fits** **4**
3.1 Logic Programming — Prolog Inference Engine . . . 4
3.1.1 What It Does . . . 4
3.1.2 Core Knowledge Base Structure . . . 4
3.1.3 Why Logic Programming Fits Here . . . 5
3.1.4 Integration with Python via PySwip . . . 6
3.2 Object-Oriented Programming — Django REST API . . . 6
3.2.1 What It Does . . . 6
3.2.2 Why OOP Fits Here . . . 7
3.3 Functional / Declarative Programming — Prompt Builder and AI Client . 7
3.3.1 What It Does . . . 7
3.3.2 Why This Paradigm Fits Here . . . 8
3.4 Event-Driven / Reactive Programming — React Native Frontend . . . 8
3.4.1 What It Does . . . 9
3.4.2 Why Event-Driven Programming Fits Here . . . 10
3.5 Imperative Programming — Query Execution and Control Flow . . . 10
3.6 Paradigm-Location Summary Table . . . 11

**4 Knowledge Base Design** **11**
4.1 Course Catalog . . . 11
4.2 Student Profiles . . . 12
4.3 Inference Rule Chain . . . 12

**5 Gemini AI Integration** **13**
5.1 Architecture . . . 13
5.2 Error Handling Strategy . . . 13

**6 Frontend Design** **13**
6.1 Component Hierarchy . . . 14
6.2 State Management . . . 14
6.3 UI Screenshots . . . 14

**7 Testing** **15**
7.1 Test Suite Architecture . . . 15
7.2 Test Categories and Coverage . . . 16
7.3 Key Test Scenarios . . . 16
7.4 Test Output Placeholder . . . 17

**8 API Endpoint Documentation** **17**
8.1 Sample Response Object . . . 17

**9 Reflection: Choosing an Approach for a Real AI System** **18**
9.1 The Core Trade-off: Symbolic vs. Statistical AI . . . . . 18
9.2 What We Would Keep From Each Paradigm . . . . . 18
9.2.1 Logic Programming: Keep for Constraint and Rule Enforcement . . 18
9.2.2 LLM Integration: Upgrade and Constrain . . . . . 19
9.2.3 The Architecture We Would Build . . . . . 19
9.3 Paradigm Choices in Production . . . . . 20
9.4 Why Not Pure LLM? . . . . . 20
9.5 Why Not Pure Prolog? . . . . . 20
9.6 Conclusion . . . . . 21

**10 Project File Structure** **21**

# 1. Introduction

The **Smart Study Advisor** is a full-stack course recommendation system built to demonstrate multiple programming paradigms working in concert. It recommends university courses to students based on their academic level, completed coursework, and topic interests. The system supports two recommendation engines:

- **Prolog Inference Engine** — a logic-programming backend that reasons over a structured knowledge base of courses, prerequisites, difficulty levels, and student profiles using formal inference rules.
- **Gemini AI Engine** — a large language model integration via Google's Generative Language API, providing natural-language, contextual recommendations.

The project spans three major layers: a **Prolog knowledge base** (`prolog_engine/`), a **Django REST API** (`backend/`), and a **React Native / Expo mobile frontend** (`frontend/`).

**Key Goal:** Demonstrate how different programming paradigms — logic, object-oriented, event-driven, declarative, functional, and AI-driven — can be composed into a single cohesive application, each paradigm applied where it provides the most natural and powerful fit.

# 2. System Architecture Overview

## 2.1. High-Level Architecture Diagram

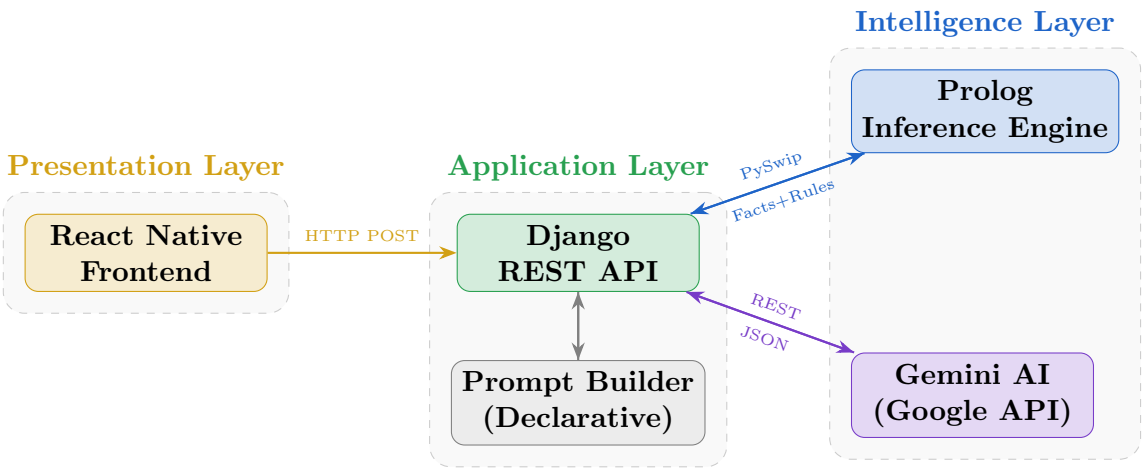



Figure 1: High-level three-layer architecture of the Smart Study Advisor system

## 2.2. Technology Stack Summary

Table 1: Technology stack and associated programming paradigms

| Layer | Technology | Paradigm(s) | Key Files |
|---|---|---|---|
| **Knowledge Base** | SWI-Prolog | Logic / Declarative | `smart_study_advisor.pl` |
| **Backend API** | Django + DRF | OOP / Imperative | `views.py`, `ai_views.py` |
| **AI Integration** | Gemini REST API | Functional / Declarative | `gemeni_client.py`, `prompt_builder.py` |
| **Frontend** | React Native / Expo | Event-Driven / Functional | `AdvisorScreen.js`, `client.js` |
| **Testing** | SWI-Prolog Harness | Logic / Declarative | `run_tests.pl` |

# 3. Architecture: Where Each Paradigm Is Used and Why It Fits

## 3.1. Logic Programming — Prolog Inference Engine

**Paradigm: Logic Programming (Prolog)**

**Location:** `prolog_engine/smart_study_advisor.pl`
**Integrated via:** `pyswip` in `backend/api/views.py`

### 3.1.1. What It Does

The Prolog knowledge base encodes the entire course recommendation domain as a set of *facts* (course difficulty, prerequisites, topic mappings, student profiles) and *rules* (eligibility, interest matching, can-take logic). The system derives recommendations purely through backward-chaining inference — no procedural code is written; the engine is simply asked queries.

### 3.1.2. Core Knowledge Base Structure

```
% FACTS: Course catalog
course(ai,                    medium).
course(algorithms,            hard).
course(web_development,       easy).

% FACTS: Prerequisites
prerequisite(ai,              algorithms).
prerequisite(ai,              discrete_math).
```

```
prerequisite(machine_learning, ai).

% FACTS: Topic mappings
course_topic(ai,              ai).
course_topic(ai,              programming).

% RULE: All prerequisites satisfied?
prerequisites_met(_Student, Course) :-
    \+ prerequisite(Course, _).
prerequisites_met(Student, Course) :-
    prerequisite(Course, _),
    \+ (prerequisite(Course, Prereq),
        \+ completed(Student, Prereq)).

% RULE: Can a student take a course?
can_take(Student, Course) :-
    course(Course, Difficulty),
    not_completed(Student, Course),
    prerequisites_met(Student, Course),
    level(Student, Level),
    suitable_difficulty(Level, Difficulty).

% RULE: Recommend courses matching interests
recommend(Student, Course) :-
    can_take(Student, Course),
    once(matches_interest(Student, Course)).
```

Listing 1: Core facts and rules in smart_study_advisor.pl

### 3.1.3. Why Logic Programming Fits Here

- **Natural domain fit:** Course advising is inherently a reasoning problem. Eligibility rules, prerequisite chains, and academic level constraints map cleanly to logical clauses without any algorithmic overhead.
- **Declarative clarity:** The knowledge and the logic are separated. Adding a new course or rule requires inserting a single fact/clause — the inference engine handles the rest automatically.
- **Built-in backtracking:** Prolog's search mechanism automatically explores all valid recommendations without explicit loops or data structures.
- **Explainability:** Each recommendation can be traced back through the rule chain, providing a transparent audit trail. This is critical in academic advisory contexts.
- **Correct representation of negation:** Prolog's \+ (negation-as-failure) naturally expresses "course not yet completed" and "prerequisites not unmet" without complex conditional logic.

### 3.1.4. Integration with Python via PySwip

The Django backend loads and queries the Prolog engine at runtime using the `pyswip` library:

```
prolog = Prolog()
prolog.consult('/path/to/smart_study_advisor.pl')

# Assert dynamic facts for the current student
prolog.assertz(f level(temp_student, {level}) )
for interest in interests:
    prolog.assertz(f prefers(temp_student, {interest}) )

# Query recommendations
results = list(prolog.query( recommend(temp_student, Course) ))

# Clean up after query
prolog.retractall( level(temp_student, _) )
```

Listing 2: Prolog integration in views.py

This pattern dynamically assembles a student's profile in the Prolog knowledge base, queries it, and then cleanly retracts those facts — ensuring session isolation.

## 3.2. Object-Oriented Programming — Django REST API

**Paradigm: Object-Oriented Programming (OOP)**

**Location:** `backend/api/views.py`, `backend/api/ai_views.py`
**Framework:** Django + Django REST Framework

### 3.2.1. What It Does

The Django REST Framework views are implemented as class-based views inheriting from `APIView`. Each HTTP verb (GET, POST) is encapsulated as a method of the class. The Prolog engine is instantiated once at module load time (as a module-level object), demonstrating the object lifecycle management.

```
class CourseRecommendationView(APIView):

    OOP: Inheriting from APIView encapsulates HTTP methods
    inside a class structure, managing state and behavior
    together.
    
    def post(self, request):
        student_prefs = request.data.get('preferences', {})
        level         = student_prefs.get('level', 'beginner')
        interests     = student_prefs.get('interests', [])
        completed_courses = student_prefs.get('
            completed_courses', [])

        # Assert facts, query, retract (Prolog integration)
```

```
        prolog.assertz(f level(temp_student, {level}) )
        results = list(prolog.query( recommend(temp_student,
            Course) ))
        prolog.retractall( level(temp_student, _) )

        return Response({
            status : success ,
            recommendations : recommendations
        }, status=status.HTTP_200_OK)
```

Listing 3: OOP-based view design in views.py

### 3.2.2. Why OOP Fits Here

- **Encapsulation:** Each view class groups HTTP logic, request validation, and response formatting into a self-contained unit. The Prolog engine object and its query lifecycle are managed cleanly inside the class.
- **Inheritance and reuse:** `APIView` provides authentication, content negotiation, exception handling, and throttling for free. The view only overrides the `post` method.
- **Django's ORM:** Though not heavily used here, Django's model system (OOP-based) is ready for future student/course persistence.
- **Separation of concerns:** `CourseRecommendationView` handles Prolog-based recommendations; `AIRecommendationView` handles Gemini-based ones. Each class has a single responsibility.

## 3.3. Functional / Declarative Programming — Prompt Builder and AI Client

**Paradigm: Declarative + Functional Programming**

**Location:** `backend/api/prompt_builder.py`, `backend/api/gemeni_client.py`

### 3.3.1. What It Does

The `prompt_builder.py` module constructs structured prompts using a pure function — given a student preference dictionary as input, it returns a formatted string as output, with no side effects. The `gemeni_client.py` sends the prompt to Google's Generative Language API and extracts the response.

```
def build_prompt(preferences: dict) -> str:
    level          = preferences.get( level , intermediate )
    interests_list = preferences.get( interests , [])
    completed_list = preferences.get( completed_courses , [])

    # Declarative: transform data, no mutable state
    interests = , .join(interests_list) if interests_list
        else None specified
```

```
    completed = ,␣ .join(completed_list) if completed_list else None

    if not interests_list:
        strategy = ( Since␣the␣student␣has␣no␣specific␣interests,␣
                     prioritize␣fundamental␣and␣core␣courses. )
    else:
        strategy = f Match␣courses␣heavily␣with␣these␣interests:␣{interests}.

    return f
You␣are␣a␣university␣course␣advisor.␣Recommend␣courses␣for␣this␣student.

Student␣profile:
-␣Level:␣{level}
-␣Interests:␣{interests}
-␣Completed␣Courses:␣{completed}

Recommendation␣Strategy:␣{strategy}

Return␣ONLY␣a␣JSON␣array,␣no␣explanation,␣no␣markdown.␣Format:
[{{ course_name :␣ X ,␣ difficulty :␣ Hard ,␣ match_reason :␣ why }}]

```

Listing 4: Declarative prompt construction in prompt_builder.py

### 3.3.2. Why This Paradigm Fits Here

- **Pure function:** `build_prompt` is a stateless transformation. Same inputs always yield the same prompt, making it trivially testable and cacheable.
- **Declarative intent:** The prompt *describes* what the AI should produce rather than *instructing* it step by step — aligning naturally with declarative thinking.
- **Separation from side effects:** All I/O (HTTP calls to Google) is isolated in `gemeni_client.py`, while `build_prompt` remains a pure data transformation module. This functional decomposition makes testing straightforward.
- **Composability:** The prompt builder can be independently swapped, mocked, or extended without touching the API client or the view layer.

## 3.4. Event-Driven / Reactive Programming — React Native Frontend

**Paradigm: Event-Driven + Functional (Hooks) Programming**

**Location:** `frontend/src/screens/AdvisorScreen.js`, `frontend/src/components/`

### 3.4.1. What It Does

The React Native frontend is entirely event-driven. User interactions (button presses, switch toggles, chip selections) trigger state changes via React hooks, which reactively re-render the UI. All components are functional (no class components), leveraging React's `useState` hook for local state management.

```
export const AdvisorScreen = () => {
  // State declarations (React hooks - functional paradigm)
  const [level, setLevel]        = useState('intermediate');
  const [interests, setInterests] = useState([]);
  const [completed, setCompleted] = useState([]);
  const [isAiMode, setIsAiMode]   = useState(false);
  const [recommendations, setRecommendations] = useState(null);
  const [loading, setLoading]    = useState(false);
  const [error, setError]        = useState(null);

  // Event handler: user toggles a chip (interest or course)
  const toggleArrayItem = (item, state, setState) => {
    setState(state.includes(item)
      ? state.filter(i => i !== item)
      : [...state, item]
    );
  };

  // Async event handler: fetch recommendations on button press
  const handleFetch = async () => {
    setLoading(true);
    setError(null);
    try {
      const data = await fetchCourseRecommendations({
        use_ai: isAiMode,
        preferences: { level, interests, completed_courses:
            completed }
      });
      setRecommendations(data.recommendations || []);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  // Declarative JSX: UI is a function of state
  return (
    <SafeAreaView style={styles.safeArea}>
      <Switch value={isAiMode} onValueChange={setIsAiMode} />
      <TouchableOpacity onPress={handleFetch}>
        <Text>Generate Syllabus</Text>
      </TouchableOpacity>
    </SafeAreaView>
  );
```

```
};
```

Listing 5: Event-driven state management in AdvisorScreen.js

### 3.4.2. Why Event-Driven Programming Fits Here

- **Asynchronous by nature:** Mobile user interactions are inherently asynchronous — the UI must remain responsive while network calls complete. React's event model with `async/await` handles this elegantly.
- **Reactive UI:** React's unidirectional data flow (state → render) ensures the UI always reflects the current state, preventing inconsistencies that imperative DOM manipulation introduces.
- **Component reusability:** The `Chip`, `FormSection`, and `CourseList` components are pure functions of their props, enabling reuse across the app without shared mutable state.
- **Declarative rendering:** JSX describes *what* the UI should look like for a given state, not *how* to update it imperatively. This greatly reduces the cognitive load of UI programming.
- **Natural fit for mobile:** Expo and React Native's event loop integrates with native touch events, gestures, and the device's rendering pipeline without exposing low-level threading.

## 3.5. Imperative Programming — Query Execution and Control Flow

**Paradigm: Imperative Programming**

**Location:** `backend/api/views.py` (the `post` method body)

The sequential steps inside the Django `post` method — extracting request data, asserting Prolog facts, executing queries, building the response list, retracting facts — are imperative in nature. This is appropriate because:

- The order of operations is critical (facts must be asserted before querying, retracted after).
- Error handling via `try/finally` ensures cleanup even on exceptions.
- Imperative code here is minimal and scoped tightly, with the heavy logic delegated to Prolog or the AI client.

### 3.6. Paradigm-Location Summary Table

Table 2: Complete mapping of paradigms to system components

| Paradigm | Component | Key Mechanism | Why It Fits |
|---|---|---|---|
| Logic / Declarative | Prolog Engine | Facts, rules, backward-chaining | Domain is inherently relational; backtracking finds all valid recommendations |
| Object-Oriented | Django Views | Class inheritance, encapsulation | Organises HTTP concerns; clean lifecycle for Prolog engine object |
| Functional / Declarative | Prompt Builder, AI Client | Pure functions, no side effects | Prompt construction is a data transformation; isolation enables testability |
| Event-Driven / Reactive | React Native Frontend | Hooks, callbacks, async/await | Mobile UIs are inherently event-based; reactive state ensures consistency |
| Imperative | View method bodies | Sequential statements, try/finally | Order-dependent operations (assert → query → retract) need explicit control flow |
| Declarative (Schema) | Django Settings, URLs | Configuration over code | URL routing and middleware stack declared, not programmed |

## 4. Knowledge Base Design

### 4.1. Course Catalog

The knowledge base encodes 15 courses across three difficulty tiers and nine topic domains:

Table 3: Course catalog with difficulty and topic mappings

| Course | Difficulty | Topics |
|---|---|---|
| Intro Programming | Easy | programming |
| Discrete Math | Easy | math |
| Databases | Easy | data |
| Digital Logic | Easy | systems |
| Web Development | Easy | web, programming |
| Data Structures | Medium | programming, math |
| Operating Systems | Medium | systems |
| AI | Medium | ai, programming |
| Computer Networks | Medium | networks, systems |
| Software Engineering | Medium | programming, design |
| Algorithms | Hard | math, programming |
| Machine Learning | Hard | ai, math |
| Computer Graphics | Hard | math, design |
| Cybersecurity | Hard | security, networks |
| Compilers | Hard | programming, systems |

## 4.2. Student Profiles

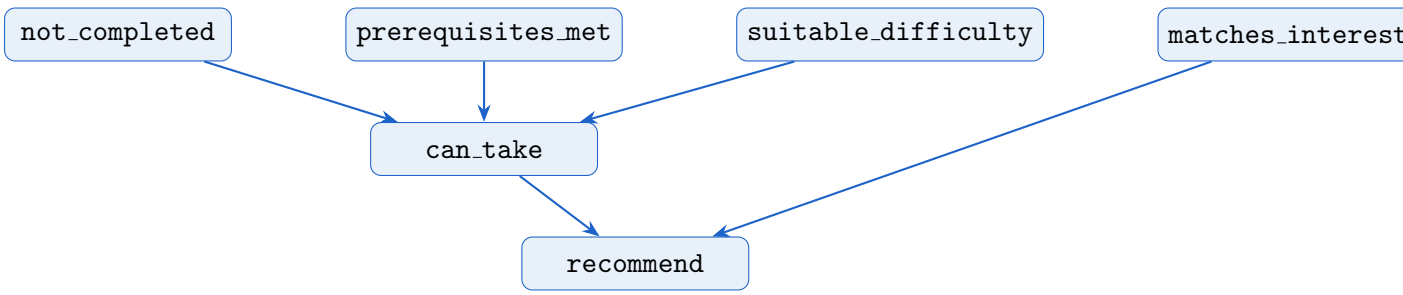
Three pre-defined student profiles cover the full spectrum of academic levels:

Table 4: Pre-defined student profiles in the knowledge base

| Student | Level | Interests | Completed |
|---|---|---|---|
| Omar | Intermediate | AI, math, programming | Intro, Discrete Math, Data Structures, Algori |
| Sara | Beginner | Web, design, programming | Intro Programming |
| Ali | Advanced | Security, networks, systems | 9 courses including Networks, OS, Algorithms |

## 4.3. Inference Rule Chain

The recommendation logic follows a clear inference chain:




A course is **recommended** to a student only if: (1) they have not completed it, (2) all prerequisites are satisfied, (3) the difficulty matches their level, and (4) at least one course topic aligns with their interests.

# 5. Gemini AI Integration

## 5.1. Architecture

The AI recommendation pipeline consists of three stages:

1. **Prompt Construction** (`prompt_builder.py`): A pure function assembles a structured prompt embedding the student's level, interests, completed courses, and a curated list of available courses. The prompt instructs Gemini to return *only* a JSON array.

2. **API Call** (`gemeni_client.py`): The prompt is sent to `gemini-flash-latest` via a POST request with a `temperature` of 0.8 for controlled creativity. Timeouts and HTTP errors are caught and re-raised as typed exceptions.

3. **Response Parsing** (`ai_views.py`): The raw response is stripped of any markdown fences, then parsed as JSON. Malformed responses return a 502 error; API failures return a 503 error.

## 5.2. Error Handling Strategy

```
cleaned = raw.strip()
if cleaned.startswith( ``` ):
    lines = cleaned.split('\n')
    lines = lines[1:] if lines[0].startswith( ``` ) else lines
    lines = lines[:-1] if lines and lines[-1].startswith( ``` ) else lines
    cleaned = '\n'.join(lines).strip()

recommendations = json.loads(cleaned)  # Raises JSONDecodeError on failure
```

Listing 6: Robust response parsing in ai_views.py

The dual-engine architecture (Prolog + Gemini) gives the user the choice between deterministic, explainable logic-based recommendations and flexible, context-aware AI-driven suggestions.

# 6. Frontend Design

## 6.1. Component Hierarchy

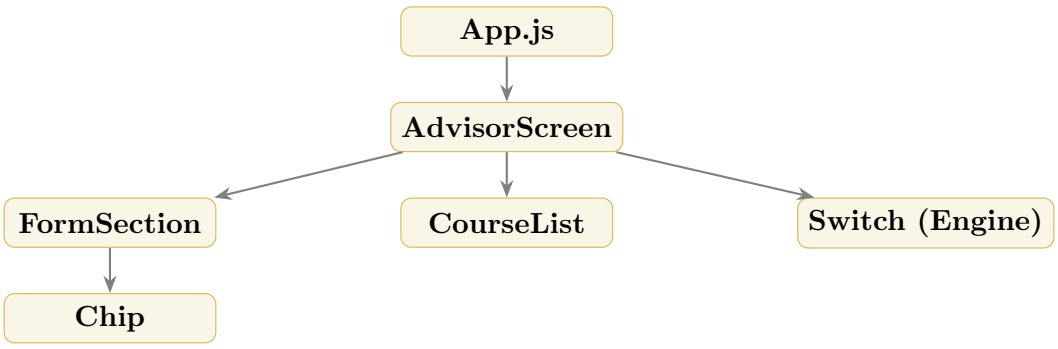
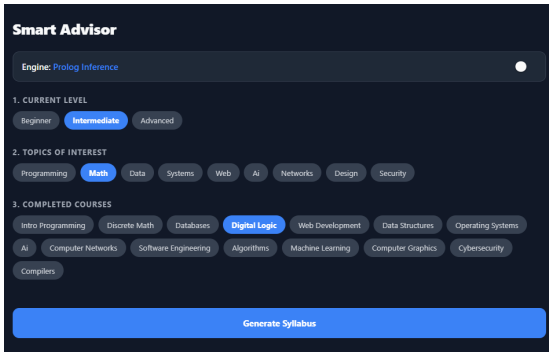



Figure 2: React Native component hierarchy

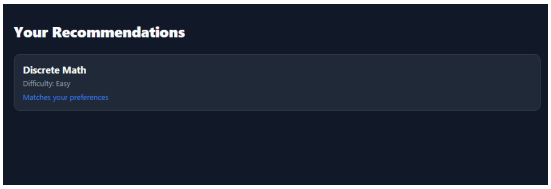
## 6.2. State Management

All state lives in `AdvisorScreen` and flows downward as props. The seven state variables cover: selected level, interest chips, completed course chips, AI/Prolog mode toggle, recommendation results, loading state, and error messages.

## 6.3. UI Screenshots




(a) Preference selection screen




(b) Recommendation results screen

Figure 3: Smart Study Advisor mobile UI (React Native / Expo)

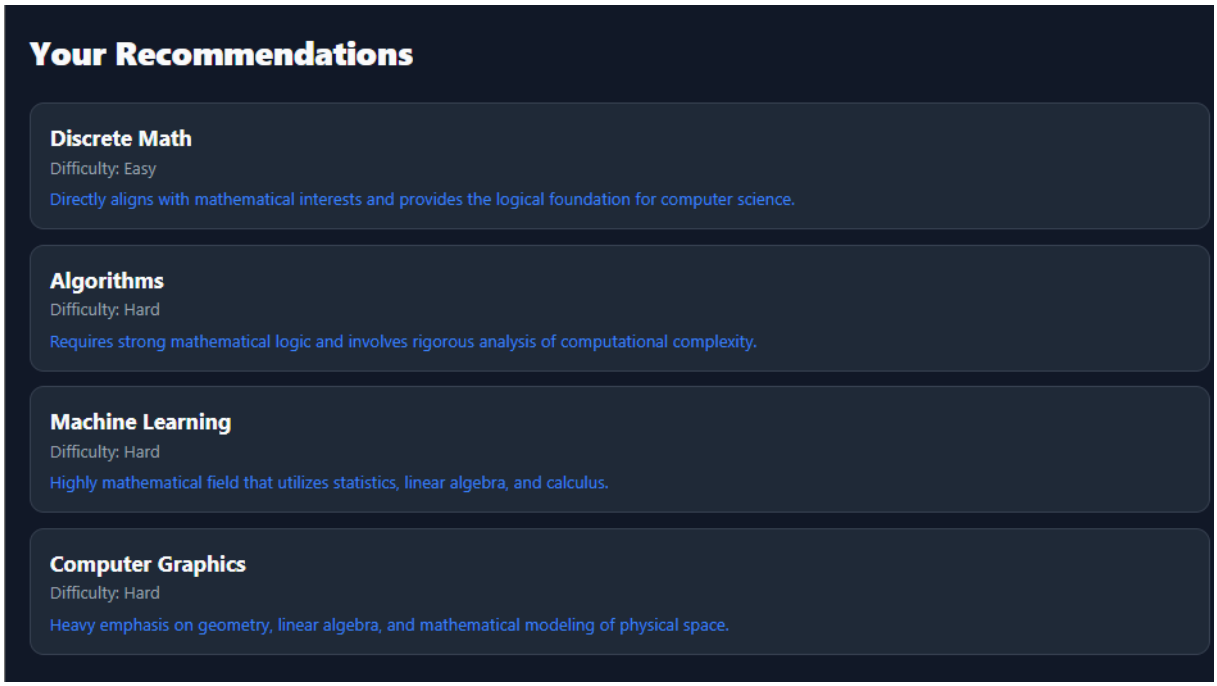



Figure 4: AI mode (Gemini) recommendation output

# 7. Testing

## 7.1. Test Suite Architecture

The Prolog test suite (`run_tests.pl`) implements a lightweight test harness entirely in Prolog, demonstrating the language's self-reflective capabilities. It maintains pass/fail counters via dynamic predicates and evaluates 30+ assertions across 10 test groups.

```
:- dynamic t_total/1, t_pass/1, t_fail/1.

check_true(Label, Goal) :-
    ( call(Goal) -> pass(Label) ; fail(Label, false, true) ).

check_eq(Label, Result, Expected) :-
    ( Result == Expected -> pass(Label)
    ; fail(Label, Result, Expected) ).
```

Listing 7: Test harness core in run_tests.pl

## 7.2. Test Categories and Coverage

Table 5: Test suite coverage breakdown

| Group | Category | Tests | Focus |
|---|---|---|---|
| 1 | Course Facts | 5 | Catalog completeness, data integrity |
| 2 | Prerequisites | 5 | Prerequisite chain correctness |
| 3 | Student Profiles | 8 | Level, completed, preferences facts |
| 4 | not_completed/2 | 3 | Negation-as-failure correctness |
| 5 | prerequisites_met/2 | 6 | Prerequisite satisfaction logic |
| 6 | matches_interest/2 | 5 | Topic-to-interest alignment |
| 7 | can_take/2 | 7 | Combined eligibility check |
| 8 | recommend/2 | 6 | End-to-end recommendation predicate |
| 9 | all_recommendations/2 | 3 | Aggregate recommendation lists |
| 10 | all_takeable/2 | 4 | Takeable course set verification |
| **Total** | | **52+** | |

## 7.3. Key Test Scenarios

Table 6: Representative test cases

| Test Query | Expected | Reasoning |
|---|---|---|
| `can_take(sara, ai)` | `false` | Sara lacks algorithms + discrete_math prerequisites |
| `can_take(omar, ai)` | `true` | Omar has all prerequisites; intermediate + medium = suitable |
| `can_take(omar, algorithms)` | `false` | Omar has already completed algorithms |
| `recommend(ali, web_development)` | `false` | Ali can take it but web doesn't match his interests |
| `all_recommendations(omar, R)` | `[ai, software_engineering, web_development]` | Sorted eligible + interest-matching courses |

### 7.4. Test Output Placeholder

[PLACEHOLDER]

Insert terminal screenshot of test run output:

```
swipl -g "run_tests, halt" run_tests.pl
```

Expected output format:

```
[ 1.  Course Facts ]
PASS  course(ai, medium) exists
PASS  15 courses in knowledge base
...
Results:  52 / 52 passed -- ALL TESTS PASSED
```

Figure 5: Expected Prolog test suite output (`run_tests.pl`)

## 8. API Endpoint Documentation

Table 7: REST API endpoints

| Method | Endpoint | Request Body | Response |
|---|---|---|---|
| POST | /api/recommend/ | {"preferences": {"level": "intermediate", "interests": ["ai"], "completed_courses": ["algorithms"]}} | {"status": "success", "recommendations": [{...}]} |
| POST | /api/ai-recommend/ | {"preferences": {...}} | {"status": "success", "message": "...", "recommendations": [{...}]} |

### 8.1. Sample Response Object

```
{
  status : success ,
  message : Recommendations generated by Prolog engine. ,
  recommendations : [
```

```
    {
        course_name : Artificial Intelligence ,
        difficulty : Medium ,
        match_reason : Matches your preferences
    },
    {
        course_name : Software Engineering ,
        difficulty : Medium ,
        match_reason : Matches your preferences
    }
  ]
}
```

Listing 8: Sample API response

# 9. Reflection: Choosing an Approach for a Real AI System

**Reflection**

If you were building a **real, production-grade AI course advisory system**, what approach would you choose and why?

## 9.1. The Core Trade-off: Symbolic vs. Statistical AI

This project presents the fundamental tension in modern AI system design: **symbolic / logic-based reasoning** (Prolog) versus **statistical / neural reasoning** (LLMs). Each has profound strengths and limitations that determine where it belongs in a real system.

## 9.2. What We Would Keep From Each Paradigm

### 9.2.1. Logic Programming: Keep for Constraint and Rule Enforcement

In a real system, the Prolog engine — or its equivalent in a modern rule engine like Drools, or a constraint satisfaction framework — would remain as the **guardrail layer**. The reasons are compelling:

- **Hard constraints must be enforced deterministically.** No LLM should be trusted to reliably respect prerequisite chains, credit requirements, or enrollment caps. These are binary — either satisfied or not — and Prolog's inference guarantees correctness.
- **Auditability and explainability.** University advisors and accreditation bodies require traceable recommendations. A Prolog trace can generate a step-by-step proof of why a course was or was not recommended — something no neural model can provide natively.

- **Domain stability.** Course prerequisites and academic rules change slowly. Maintaining a Prolog knowledge base for this is low-cost and high-reliability.

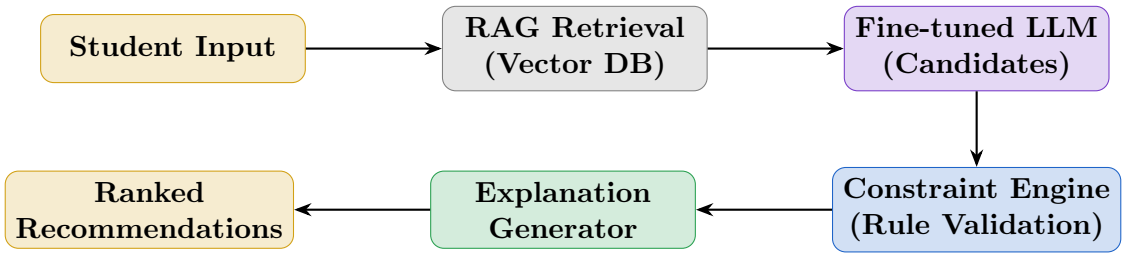
However, raw Prolog would be replaced with a **more scalable knowledge representation** (e.g., a graph database like Neo4j encoding courses as nodes and prerequisites as edges), with a rule engine operating over it.

### 9.2.2. LLM Integration: Upgrade and Constrain

The Gemini integration in this project is effective as a prototype, but a production system would require:

- **Retrieval-Augmented Generation (RAG):** Rather than embedding the full course list in the prompt, a vector database (e.g., Pinecone, Weaviate) would store rich course descriptions, syllabi, and past student reviews. The LLM retrieves relevant context before generating recommendations, drastically improving specificity and reducing hallucination.
- **Output filtering:** All LLM recommendations would be post-processed through the constraint layer. If the LLM suggests a course for which the student lacks prerequisites, it is silently excluded — the LLM provides *candidates*, the logic engine provides *validation*.
- **Fine-tuning on institutional data:** A production system would fine-tune a smaller, faster model (e.g., Llama 3 or Mistral) on historical advising data from the institution, capturing nuanced patterns (e.g., "students who liked Algorithms tend to enjoy Compilers") that static rules cannot encode.

### 9.2.3. The Architecture We Would Build



## 9.3. Paradigm Choices in Production

Table 8: Production paradigm selection rationale

| Concern | Chosen Approach | Rationale |
|---|---|---|
| Constraint enforcement | Logic programming / Rule engine | Deterministic, auditable, legally defensible |
| Personalisation | Fine-tuned LLM + RAG | Captures nuanced patterns at scale |
| Scalability | Microservices (Python/Go) | Each service independently scalable |
| Explainability | Prolog proof traces + LLM justifications | Dual-layer: formal + natural-language |
| Data freshness | Graph DB + embedding pipeline | Courses, syllabi, reviews updated continuously |
| Mobile UI | React Native (retained) | Mature, cross-platform, event-driven |

## 9.4. Why Not Pure LLM?

A common temptation would be to replace everything with an LLM. This would be a mistake in an academic advisory context:

- **Hallucination risk:** LLMs can confidently recommend a course a student is not qualified for. In an academic context, this causes real harm (failed registrations, delayed graduation).
- **Non-determinism:** The same query on two different days might produce different recommendations, making the system unreliable and difficult to audit.
- **No formal guarantees:** Prerequisite chain enforcement requires logical completeness that probabilistic models cannot provide.

## 9.5. Why Not Pure Prolog?

Conversely, pure logic programming at scale has its own limitations:

- **Knowledge acquisition bottleneck:** Every rule must be hand-coded. Encoding "students who enjoyed Discrete Math tend to also enjoy Algorithms" requires explicit rules, while an LLM learns this from data.
- **Natural language:** Prolog cannot process free-text student goals like "I want to build mobile apps." An LLM maps this intent to relevant topics naturally.
- **Context sensitivity:** Career goals, learning style preferences, and semester-specific availability require soft, contextual reasoning that rules alone cannot capture.

### 9.6. Conclusion

The ideal production system is a **neurosymbolic hybrid**: statistical learning for preference modelling and natural language understanding, combined with symbolic reasoning for constraint enforcement and explainability. The architecture built in this lab is a strong conceptual prototype of exactly this pattern — a Prolog engine for deterministic rules and an LLM for flexible reasoning, connected by a clean API layer. The production version would deepen each component (RAG, fine-tuning, graph KB) and add the infrastructure (caching, monitoring, A/B testing, feedback loops) required for institutional deployment.

**The core insight remains:** not all problems benefit from the same paradigm. Choosing the right tool for each sub-problem — logic for rules, neural models for patterns, functional design for data pipelines, event-driven architecture for user interaction — is the hallmark of mature software engineering.

## 10. Project File Structure

```
smart-study-advisor/
 prolog_engine/
    smart_study_advisor.pl   # Knowledge base + inference rules
    run_tests.pl             # Automated test suite
    test_queries.pl          # Manual query documentation

 backend/
    api/
       views.py             # Prolog-based recommendation view (OOP)
       ai_views.py          # Gemini AI recommendation view (OOP)
       prompt_builder.py    # Prompt construction (Functional)
       gemeni_client.py     # Gemini API client (Functional)
       urls.py              # URL routing (Declarative)
       advisor.pl           # Placeholder Prolog stub
    smart_advisor/
       settings.py          # Django configuration (Declarative)
    requirements.txt         # Python dependencies

 frontend/
    src/
       screens/
          AdvisorScreen.js # Main screen (Event-Driven)
       components/
          Chip.js          # Selectable chip component
          FormSection.js   # Form section wrapper
          CourseList.js    # Results display component
       api/
          client.js        # Axios API client (Functional)
       theme/
          colors.js        # Design tokens (Declarative)
       utils/
           constants.js     # Domain constants
```

```
    App.js                    # Root component
    package.json

README.md
```